XChain 핵심 타입 및 데이터 구조 분석

개요

`lib/xchain` 패키지는 Omni 네트워크의 크로스체인 프로토콜에서 사용되는 핵심 타입과 인터페이스를 정의합니다. 이 패키지는 크로스체인 메시징, 블록 증명, 검증자 서명 등 크로스체인 통신의 모든 기본 요소를 포함합니다.

핵심 개념

1. 확인 레벨 (ConfLevel)

확인 레벨은 블록의 "버전"을 정의하며, 최종화된 블록과 임시(fuzzy) 블록을 구분합니다.

```
type ConfLevel byte


const (
ConfUnknown ConfLevel = 0  // unknown
ConfLatest ConfLevel = 1  // latest
ConfFinalized ConfLevel = 4  // final
ConfMin1 ConfLevel = 5  // min1
ConfMin2 ConfLevel = 6  // min2
ConfConfirmed ConfLevel = 7  // confirmed (Solana 전용)
)
```

주요 메서드:

- `Valid()` : 확인 레벨이 유효한지 검증
- `IsFuzzy()` : 최종화되지 않은 임시 상태인지 확인
- `IsFinalized()` : 최종화된 상태인지 확인
- `Label()` : 확인 레벨의 짧은 라벨 반환
- `MinX()` : ConfMinX 레벨에서 X 값 추출

확인 레벨 분류:

- **최종화된 블록**: `ConfFinalized` - 되돌릴 수 없는 확정된 블록
- **임시 블록**: `ConfLatest` , `ConfMin1` , `ConfMin2` , `ConfConfirmed` - 아직 최종화되지 않은 블록

2. 체인 버전 (ChainVersion)

체인의 특정 상태를 식별하는 구조체입니다.

```
type ChainVersion struct {
ID  uint64  // 체인 ID (https://chainlist.org/ 기준)
ConfLevel ConfLevel // 블록 확인 레벨
}
```

3. 샤드 ID (ShardID)

스트림을 세분화하여 관리하는 식별자입니다.

```
type ShardID uint64
```

```
const (
ShardFinalized0 = ShardID(ConfFinalized) // 기본 최종화 샤드
ShardLatest0 = ShardID(ConfLatest) // 기본 최신 샤드
ShardBroadcast0 = ShardID(ConfFinalized) | 0x0100  // 브로드캐스트 샤드
)
```

주요 메서드:

- `ConfLevel()` : 샤드에 인코딩된 확인 레벨 추출
- `Flags()` : 샤드의 플래그 바이트 추출
- `Broadcast()` : 브로드캐스트 플래그 확인
- `Label()` : 샤드의 짧은 라벨 반환

메시지 관련 타입

1. 스트림 ID (StreamID)

크로스체인 스트림을 고유하게 식별합니다.

```
type StreamID struct {
SourceChainID  uint64  // 소스 체인 ID
DestChainID  uint64  // 목적지 체인 ID
ShardID ShardID // 샤드 ID (확인 레벨과 매핑)
}
```

주요 메서드:

- `ConfLevel()` : 스트림의 확인 레벨 반환

- `ChainVersion()` : 스트림의 체인 버전 반환

2. 메시지 ID (MsgID)

크로스체인 메시지를 고유하게 식별합니다.

```
type MsgID struct {
StreamID // 메시지가 속한 스트림 ID
StreamOffset  uint64  // 스트림 내 메시지의 순차적 오프셋 (1부터 시작)
}
```

주요 메서드:

- `Hash()` : RouteScan 크로스 트랜잭션 ID 해시 생성

3. 크로스체인 메시지 (Msg)

실제 크로스체인 메시지 데이터를 포함합니다.

```
type Msg struct {
MsgID // 메시지 고유 ID
SourceMsgSender common.Address // 소스 체인의 발신자 주소
DestAddress common.Address // 목적지 체인의 대상 주소
Data []byte  // 호출할 데이터
DestGasLimit  uint64  // 목적지에서 사용할 가스 한도
TxHash common.Hash // 소스 체인 트랜잭션 해시
Fees *big.Int // 크로스체인 호출 수수료
LogIndex  uint64  // 블록 내 이벤트 로그 인덱스
```

```
}
```

4. 메시지 영수증 (Receipt)

크로스체인 메시지 실행 결과를 나타냅니다.

```
type Receipt struct {
MsgID // 실행된 메시지 ID
GasUsed  uint64  // 사용된 가스량
Success  bool  // 실행 성공 여부
Error []byte  // 실패 시 오류 메시지
RelayerAddress common.Address // 메시지를 제출한 릴레이어 주소
TxHash common.Hash // 릴레이어 제출 트랜잭션 해시
LogIndex  uint64  // 블록 내 이벤트 로그 인덱스
}
```

블록 관련 타입

1. 블록 헤더 (BlockHeader)

크로스체인 블록을 고유하게 식별합니다.

```
type BlockHeader struct {
ChainID  uint64  // 소스 체인 ID
BlockHeight  uint64  // 소스 체인 블록 높이
```

```
BlockHash common.Hash // 소스 체인 블록 해시
}
```

2. 크로스체인 블록 (Block)

소스 체인 EVM 블록의 크로스체인 속성을 결정론적으로 표현합니다.

```
type Block struct {
BlockHeader
Msgs []Msg // 블록에서 발생한 모든 크로스체인 메시지
Receipts []Receipt // 블록에서 적용된 모든 메시지 영수증
ParentHash common.Hash // 부모 블록 해시
Timestamp time.Time // 소스 체인 블록 타임스탬프
}
```

주요 메서드:

- `ShouldAttest(attestInterval uint64)` : 블록이 증명되어야 하는지 판단
- `MsgByID(msgID MsgID)` : ID로 메시지 검색
- `ReceiptByID(msgID MsgID)` : ID로 영수증 검색

증명 조건:

1. 크로스체인 메시지가 포함된 블록
2. 증명 간격(attestInterval)에 따른 주기적 블록

증명 관련 타입

1. 증명 헤더 (AttestHeader)

쿼럼 투표가 필요한 증명을 고유하게 식별합니다.

```
type AttestHeader struct {
ConsensusChainID  uint64  // Omni 합의 체인 ID (재생 공격 방지)
ChainVersion ChainVersion // 증명 대상 체인 버전
AttestOffset  uint64  // 체인 버전별 증명 오프셋 (1부터 시작)
}
```

2. 투표 (Vote)

검증자의 크로스체인 블록에 대한 투표입니다.

```
type Vote struct {
AttestHeader // 투표가 포함될 증명 식별자
BlockHeader // 투표 대상 크로스체인 블록
MsgRoot common.Hash // 블록 내 모든 메시지의 머클 루트
Signature SigTuple // 검증자 서명과 공개키
}
```

3. 증명 (Attestation)

검증자 세트의 쿼럼 투표를 포함한 크로스체인 블록 증명입니다.

```
type Attestation struct {
```

```
AttestHeader // 증명 고유 식별자

BlockHeader // 증명 대상 크로스체인 블록

MsgRoot common.Hash // 블록 내 모든 메시지의 머클 루트

Signatures []SigTuple // 검증자 서명들과 공개키들

ValidatorSetID  uint64  // 증명을 승인한 검증자 세트 ID

}
```

주요 메서드:

- `AttestationRoot()` : 증명 루트 해시 계산

4. 서명 튜플 (SigTuple)

검증자 서명과 주소를 묶은 구조체입니다.

```
type SigTuple struct {

ValidatorAddress common.Address // 검증자 이더리움 주소

Signature Signature65 // 검증자 서명 (65바이트 [R || S || V] 형식)

}
```

5. 서명 형식 (Signature65)

이더리움 표준 65바이트 서명 형식입니다.

```
type Signature65 [65]byte  // [R || S || V] 형식
```

제출 관련 타입

1. 제출 (Submission)

크로스체인 메시지 세트와 증명을 포함한 제출 데이터입니다.

```
type Submission struct {
AttestationRoot common.Hash // 크로스체인 블록의 증명 머클 루트
ValidatorSetID uint64 // 증명을 승인한 검증자 세트 ID
AttHeader AttestHeader // 제출이 속한 증명 헤더
BlockHeader BlockHeader // 크로스체인 블록 헤더
Msgs []Msg // 제출할 메시지들
Proof [][32]byte // 메시지들의 머클 다중 증명
ProofFlags []bool // 증명이 왼쪽인지 오른쪽인지 나타내는 플래그
Signatures []SigTuple // 검증자 서명들과 공개키들
DestChainID uint64 // 목적지 체인 ID (내부 사용)
}
```

커서 관련 타입

1. 제출 커서 (SubmitCursor)

목적지 포털 컨트랙트에서 크로스체인 스트림의 진행 상황을 추적합니다.

```
type SubmitCursor struct {
StreamID // 커서가 속한 스트림 ID
MsgOffset uint64 // 스트림의 최신 제출된 메시지 오프셋
```

```
AttestOffset uint64 // 최신 제출된 크로스체인 증명 오프셋
}
```

2. 발행 커서 (EmitCursor)

소스 포털 컨트랙트에서 크로스체인 스트림의 진행 상황을 추적합니다.

```
type EmitCursor struct {
StreamID // 커서가 속한 스트림 ID
MsgOffset uint64 // 스트림의 최신 발행된 메시지 오프셋
}
```

특수 상수

브로드캐스트 체인 ID

```
const BroadcastChainID uint64 = 0 // 브로드캐스트 메시지에 사용되는 체인 ID
```

데이터 플로우

1. 메시지 생성 플로우

1. **메시지 발행**: 소스 체인에서 `Msg` 생성
2. **스트림 식별**: `StreamID` 로 메시지 경로 결정
3. **메시지 ID 할당**: `MsgID` 로 고유 식별

4. **블록 포함**: `Block` 에 메시지 포함

5. **커서 업데이트**: `EmitCursor` 로 진행 상황 추적

2. 증명 생성 플로우

1. **블록 증명 필요성 판단**: `Block.ShouldAttest()` 호출

2. **검증자 투표**: 각 검증자가 `Vote` 생성

3. **쿼럼 달성**: 충분한 투표 수집 후 `Attestation` 생성

4. **증명 루트 계산**: `AttestationRoot()` 호출

3. 메시지 제출 플로우

1. **제출 데이터 준비**: `Submission` 구조체 생성

2. **머클 증명 포함**: 메시지 검증을 위한 증명 첨부

3. **목적지 체인 제출**: 릴레이어가 메시지 제출

4. **영수증 생성**: 실행 결과로 `Receipt` 생성

5. **커서 업데이트**: `SubmitCursor` 로 진행 상황 추적

보안 고려사항

1. 재생 공격 방지

- `AttestHeader.ConsensusChainID` : 합의 체인 ID로 재생 공격 방지
- `AttestOffset` : 순차적 오프셋으로 중복 증명 방지

2. 서명 검증

- `Signature65` : 이더리움 표준 서명 형식 사용
- `SigTuple` : 검증자 주소와 서명 묶음으로 위조 방지

3. 메시지 무결성

- `MsgRoot` : 머클 루트로 메시지 집합 무결성 보장
- `Proof` : 머클 증명으로 개별 메시지 검증

이러한 타입 구조는 Omni 네트워크의 크로스체인 통신에서 안전하고 효율적인 메시지 전송을 보장하는 핵심 기반을 제공합니다.